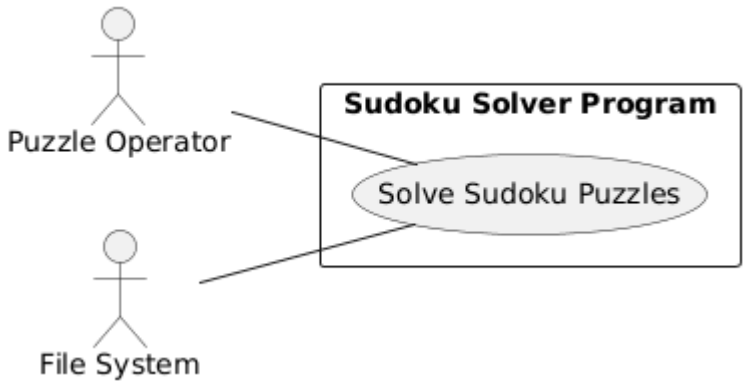
a. Identify/discover a use case

- Use case: Solve Sudoku Puzzles
  Primary actor: Puzzle Operator — the person who runs the program
  Supporting actor: File System — provides the puzzle files to the program
  Goal: The operator wants to see the solutions, or a no-solution message, for the supplied Sudoku puzzles.
- Use case diagram:




b. Brief description

This use case allows a Puzzle Operator to run the program, which reads the five puzzle files, checks and solves each puzzle, and displays each puzzle's solution or a message indicating that no solution was found.

c. Step-by-step flow (outline)

1. The Puzzle Operator starts the program.
2. The program selects the first puzzle file.
3. The program requests the puzzle file from the File System.
4. The File System provides the file.
5. The program checks that the file contains a valid puzzle.
6. The program displays the original puzzle.
7. The program checks the puzzle's starting clues for conflicts.
8. The program searches for solutions.
9. The program displays each solution it finds, or displays "No solution found."
10. The program displays the number of solutions found.

11. The program repeats steps 2–10 for the remaining puzzle files.
12. The program finishes after processing all five files.

d. Detailed description

Use case: Solve Sudoku Puzzles

Preconditions

- The Puzzle Operator can start the program.
- The expected puzzle files are available in the locations the program uses.

Basic flow

1. Operator starts the program.
   The Puzzle Operator runs the Sudoku Solver Program.
2. Program selects a puzzle file.
   The program selects the next file in its set of five puzzle files.

3. Program reads the file.
   The program requests the selected file from the File System and reads the puzzle's cells.

4. Program validates the file format.
   The program checks that the file has exactly 81 cells and that each cell contains either a digit from 1 through 9 or an underscore representing an empty cell.

5. Program displays the original puzzle.
   If the file is valid, the program displays the puzzle to the Puzzle Operator.

6. Program checks the starting clues.
   The program checks that the puzzle's given numbers do not conflict with Sudoku rules.

7. Program searches for solutions.
   If the starting clues are valid, the program searches for valid ways to complete the puzzle.

8. Program displays the result.
   The program displays each solution it finds and reports the total number of solutions. If it finds none, it displays “No solution found.”

9. Program processes the next file.
   The program repeats the file-reading and solving process for the next puzzle.

10. Program completes the run.
    After processing all five files, the program exits.

Alternative courses of action

- A1. A puzzle file cannot be opened. The program displays an error identifying the file, skips that puzzle, and continues with the next file.
- A2. A puzzle file has missing, extra, or invalid cells. The program displays a format error identifying the file, skips that puzzle, and continues with the next file.
- A3. The starting clues conflict. The program displays “No solution found” for that puzzle and continues with the next file.
- A4. The puzzle has no valid completion. The program displays “No solution found” and continues with the next file.
- A5. The puzzle has more than one valid completion. The program displays each solution it finds, reports the total, and continues with the next file.

Postcondition

The program has attempted to process all five puzzle files and displayed the result for each file it could read.